

Bàn về kỹ thuật gỡ lỗi chương trình

Xu Chuan

Nguồn gốc: On program debugging techniques

IOI China candidate team papers / 2000 / Xu Chuan.pdf

Từ khóa. Kỹ thuật gỡ lỗi; phương pháp kiểm thử; thiết kế ca kiểm thử.

Tóm tắt

Bài viết này kết hợp kinh nghiệm của tác giả để trình bày và tổng kết khá chi tiết các kỹ thuật gỡ lỗi chương trình trong thi đấu. Sau khi giới thiệu các loại lỗi thường gặp khi lập trình và các công cụ gỡ lỗi của môi trường tích hợp, bài viết đưa ra một quy trình gỡ lỗi tổng quát, rồi tập trung nói về kỹ thuật tìm lỗi động và rút ra một số quy luật. Cuối cùng, một ví dụ gỡ lỗi cụ thể được dùng để thể hiện cách áp dụng các kỹ thuật đã bàn.

1 Sự cần thiết của gỡ lỗi chương trình

Trong quá trình thiết kế chương trình, lỗi là điều khó tránh. Dù có lập trình viên cho rằng một chương trình có thể đạt tới mức hoàn mỹ không tì vết, thực tế không hẳn như vậy; nếu không, đã chẳng có nhiều người bức bối với Windows. Chương trình viết trong các kỳ thi tin học tất nhiên không bao giờ đồ sộ như Windows, nhiều lắm cũng chỉ vài trăm KB, nhưng do thời gian hạn chế, chương trình của thí sinh khó tránh khỏi thiếu sót. Vì vậy, gỡ lỗi trở thành một khâu cực kỳ quan trọng. Làm thế nào để nhanh chóng và chính xác phát hiện, sửa lỗi trong thời gian gấp gáp chính là vấn đề bài viết này muốn thảo luận.

2 Quy nạp các loại lỗi thường gặp

Bình pháp Tôn Tử nói: “Biết mình biết người, trăm trận không nguy.” Với người gỡ lỗi chương trình, lỗi trong chương trình giống như kẻ địch; nếu nắm đúng tình hình của “địch” thì hiển nhiên rất có lợi. Dưới đây, ta quy nạp một số loại lỗi thường gặp và nêu cách xử lý.

2.1 Sai ý tưởng

Cần xem đó là sai thuật toán cơ bản hay chỉ thiếu chức năng. Trường hợp trước thường phải viết lại phần lớn mã; có viết lại hay không tùy thời gian có dư dả hay không. Trường hợp sau chỉ cần thêm một phần mã rồi sửa một số chỗ; khi đó phải xét toàn diện để tránh bỏ sót những nơi cần sửa.

2.2 Lỗi cú pháp

Không có gì nhiều để nói. Là một thí sinh thi tin học, ta nên nắm rất rõ cú pháp của ngôn ngữ lập trình đã chọn; ở đây không bàn thêm chi tiết.

2.3 Lỗi viết nhầm

Loại lỗi này rất đau đầu. Những lỗi viết nhầm thông thường có thể được trình biên dịch tìm ra, nhưng nếu trong một biểu thức cần dùng biến j mà lại viết nhầm thành i , trình biên dịch không phát hiện được, còn người viết khi tự tìm cũng khó thấy vì hai ký tự khá giống nhau. Việc loại bỏ loại lỗi này chỉ dựa vào hai chữ “cẩn thận”, và cụ thể có thể dùng phương pháp tìm lỗi tĩnh sẽ nói ở dưới.

2.4 Sai định dạng đầu ra

Vì các kỳ thi tin học hiện nay dùng kiểm thử hộp đen, ví dụ mất điểm do sai định dạng đầu ra xuất hiện rất nhiều. Một dấu câu, một dấu cách cũng có thể dẫn tới hời hợt sau cùng. Vì vậy, khi gỡ lỗi, trước hết phải đối chiếu định dạng đầu ra, đồng thời thiết kế thêm vài ca kiểm thử nhắm vào các định dạng đầu ra khác nhau để tránh “sẩy chân một bước, ân hận nghìn đời”.

2.5 Các lỗi khác dễ mắc khi lập trình

Ngoài những loại lỗi trên, các lỗi khác thường bắt nguồn từ việc suy xét chưa chu đáo ở chi tiết lập trình. Dưới đây chỉ liệt kê vài lỗi tương đối kín. Chỉ khi thường xuyên tổng kết và tích lũy trong luyện tập, ta mới thật sự đạt được “biết mình biết người”.

2.5.1 Biến chưa gán giá trị ban đầu

Xem đoạn chương trình sau:

```
For i:=1 to N Do  
  If A[i]>Max Then Max:=A[i];  
WriteLn(Max);
```

Ý định ban đầu của đoạn chương trình này rõ ràng là in ra số lớn nhất trong mảng A . Nhưng vì nó bỏ sót câu lệnh gán giá trị ban đầu cho Max , rất có thể số in ra không nằm trong mảng A . Nên thêm một câu $Max:=-MaxInt$; ở đầu thủ tục. Thói quen gán giá trị ban đầu cho biến trước khi dùng có thể phòng ngừa nhiều lỗi khá kín.

2.5.2 Tràn trong phép tính trung gian

Xem hai câu lệnh sau:

```
A:=1000;  
B:=A*A Div 100;
```

Trong đó A, B đều có kiểu `Integer`. Theo suy nghĩ của ta, $1000 \cdot 1000 \text{ Div } 100 = 10000$. Nhưng khi xem giá trị B , ta lại thấy B bằng 169. Nguyên nhân là khi biên dịch, Pascal luôn tính $A*A$ trước, đặt nó vào một biến trung gian, rồi mới tính kết quả cuối cùng đưa vào B . Mà $A*A$ vượt khỏi phạm vi của `Integer`; đó là gốc rễ của lỗi. Muốn Pascal báo loại lỗi này, chỉ cần bật công tắc biên dịch `Q`. Cách xử lý là dùng ép kiểu, viết thành $B:=\text{LongInt}(A)*A \text{ Div } 100$. Trình biên dịch sẽ tự động quy định biến trung gian là kiểu `LongInt`, nên không bị tràn.

2.5.3 Biến cục bộ trùng tên biến toàn cục gây lẫn khái niệm

Điều này thực ra chưa hẳn là lỗi, nhưng rất nhiều lỗi bắt nguồn từ đó. Một lỗi thường gặp là khi dùng biến toàn cục trong một thủ tục, ta quên rằng trong thủ tục ấy còn định nghĩa một biến cục bộ cùng tên, khiến chương trình thực tế không khớp với ý tưởng. Một lỗi thường gặp khác là quên định nghĩa biến cục bộ, nên trong thủ tục hoặc hàm thực ra dùng biến toàn cục; khi lỗi xuất hiện, nó thường xuất hiện ở một chỗ khác ngoài thủ tục hoặc hàm ấy, nơi cần dùng biến toàn cục, làm tăng độ khó gỡ lỗi. Vì vậy, nên hết sức tránh dùng các biến trùng tên.

2.5.4 Lẫn lộn câu lệnh If-Then-Else

Quy định của Pascal đối với câu lệnh If-Then-Else là: If-Then có thể không có Else tương ứng; Else luôn ghép với câu If-Then gần nhất. Chính điều này làm ta dễ sai khi dùng If-Else lồng nhau. Ví dụ:

```
If dieu_kien_a
Then If dieu_kien_b Then doan_ma_b
Else doan_ma_a
```

Ý định ban đầu là cho doan_ma_a chạy khi dieu_kien_a không đúng. Nhưng vì Else luôn ghép với If-Then gần nhất, Else này ghép với câu If dieu_kien_b Then; tức là doan_ma_a chỉ chạy khi dieu_kien_a đúng và dieu_kien_b sai, khác xa ý định ban đầu. Cách giải quyết là thêm một Else rỗng vào nơi cần thiết; trong ví dụ trên, cần thêm một Else rỗng sau câu If dieu_kien_b Then.

2.5.5 So sánh số thực bị sai

Khi so sánh hai số thực có bằng nhau hay không, nếu dùng trực tiếp dấu bằng thường sẽ gây lỗi. Nguyên nhân là phép toán dấu phẩy động có sai số. Cách xử lý là so sánh trị tuyệt đối của hiệu hai số với một lượng rất nhỏ tương đối; nói chung nếu $\text{abs}(a - b) < 10^{-8}$, có thể xem $a = b$.

3 Công cụ gỡ lỗi của môi trường tích hợp

Với một chiến sĩ, cần hiểu rõ tính năng của vũ khí trong tay. Với người gỡ lỗi chương trình, công cụ gỡ lỗi chính là vũ khí; nắm thành thạo công cụ và phát huy đầy đủ tính năng của nó giúp ích rất lớn cho việc nhanh chóng tìm ra lỗi và tăng tốc độ gỡ lỗi. Dưới đây là phần giới thiệu một số công cụ gỡ lỗi do môi trường tích hợp cung cấp.

Hai menu chính thường dùng khi gỡ lỗi là Run và Debug. Menu Run cung cấp nhiều cách thực thi chương trình, còn menu Debug cung cấp chức năng quan sát, sửa biến và đặt điểm dừng.

Có bốn cách thực thi chương trình:

1. Run: chạy toàn bộ chương trình (Ctrl+F9). Cách này thường dùng trong kiểm thử tổng thể. Nói chung, mỗi ca kiểm thử đều nên trước hết dùng cách này để chạy chương trình; nếu đầu ra đúng thì có thể chuyển thẳng sang ca tiếp theo, tránh tốn thời gian không cần thiết. Ngay cả khi phát hiện lỗi, nó cũng chỉ tốn hơn việc đi thẳng vào gỡ lỗi mô-đun một chút thời gian, hoàn toàn đáng làm.
2. Step over: thực thi từng bước, coi cả một thủ tục hoặc hàm là một bước (F8). Cách này thường dùng trong giai đoạn gỡ lỗi mô-đun; có thể quan sát biến thay đổi trước và sau khi mô-đun chạy để xác định mô-đun có lỗi hay không, cũng có thể dùng để bỏ qua các mô-đun đã kiểm thử xong.
3. Trace into: thực thi từng bước và đi vào bên trong thủ tục hoặc hàm (F7). Cách này thường dùng trong giai đoạn gỡ lỗi tầng dưới, để theo dõi từng bước thực thi của chương trình. Ưu điểm là để

định vị lỗi trực tiếp; nhược điểm là tốc độ gỡ lỗi chậm, đặc biệt khi lỗi nằm ở phần sau của chương trình. Vì vậy, thông thường trước hết dùng phương pháp gỡ lỗi mô-đun để thu hẹp phạm vi lỗi tối đa, rồi dùng cách thực thi thứ tư và điểm dừng để nhanh chóng bỏ qua phần không lỗi, cuối cùng mới dùng cách này để lần theo từng bước và tìm lỗi.

4. `Go to cursor`: chạy tới vị trí con trỏ (F4). Cách này được giới thiệu cuối cùng vì độ linh hoạt lớn: có thể chạy một dòng, có thể chạy nhiều dòng; có thể trực tiếp bỏ qua thủ tục hoặc hàm, cũng có thể đi vào trong thủ tục hoặc hàm. Nó có ưu điểm định vị mạnh của điểm dừng, nhưng linh hoạt hơn điểm dừng. Dừng đúng và thích hợp cách này có thể tăng tốc độ gỡ lỗi rất nhiều, điều này dựa vào kinh nghiệm gỡ lỗi phong phú. Có thể tham khảo ví dụ gỡ lỗi ở phần sau.

Trong menu Debug, hai mục thường dùng nhất là Watch và Add Watch. Hai mục này dùng để theo dõi sự thay đổi giá trị của biến và biểu thức trong quá trình chương trình thực thi, nhờ đó có thể luôn kiểm tra chúng có được xuất theo yêu cầu thuật toán hay không, có phù hợp đáp án đúng hay không. Phần lớn lỗi trong lúc gỡ lỗi chỉ cần dùng hai mục này cùng bốn cách thực thi ở trên là có thể kiểm tra ra.

Đôi khi, dù biết một mô-đun có lỗi, ta nhất thời chưa tìm được chỗ lỗi, và ba cách thực thi sau trong phần trên cũng khó định vị nhanh. Chẳng hạn với một chương trình, ta cần nó dừng lại khi chạy tới một câu lệnh nào đó và đồng thời thỏa một điều kiện nào đó. `Go to cursor` chỉ bảo đảm chương trình dừng khi chạy tới câu lệnh ấy, chứ không bảo đảm điều kiện được thỏa; khi đó ta cần dùng điểm dừng. Điểm dừng tuy không linh hoạt bằng `Go to cursor`, nhưng có một ưu thế không thể thay thế: nó có thể đặt điều kiện ngắt. Dùng mục Add Breakpoint (Ctrl+F8) có thể đặt một điểm dừng ở vị trí con trỏ hiện tại; mục Breakpoints có thể sửa điều kiện điểm dừng. Cần chú ý rằng đặt điểm dừng sẽ làm giảm rất mạnh hiệu suất chạy chương trình, nên sau khi gỡ lỗi xong nhất định phải nhớ xóa tất cả điểm dừng.

Mục Evaluate/modify (Ctrl+F4) dùng để tạm thời quan sát và sửa giá trị của biểu thức. Nếu trong quá trình gỡ lỗi cần tạm thời quan sát hoặc sửa một biểu thức nào đó, có thể mở cửa sổ Evaluate and modify để thao tác. Cách này đặc biệt thích hợp khi gỡ lỗi thủ tục hoặc hàm: ta có thể trước hết dùng `Go to cursor` chạy tới đầu một thủ tục hoặc hàm, rồi dùng Evaluate/modify thay đổi giá trị tham số để kiểm tra kết quả thực thi của thủ tục hoặc hàm trong các tình huống khác nhau. Nhưng cần chú ý rằng một thủ tục hoặc hàm thường không cô lập; nó thường cần dùng một số biến toàn cục. Vì vậy, chỉ thay đổi tham số mà không thay đổi giá trị các biến toàn cục khác có thể là không hợp lệ. Cần đặc biệt cẩn thận để tránh tình huống này.

Mục Call stack (Ctrl+F3) dùng để hiển thị trạng thái ngăn xếp hiện tại, đặc biệt thích hợp khi gỡ lỗi thuật toán đệ quy. Ta có thể dùng nó để xem tình hình truyền tham số ở từng tầng. Tuy nhiên, nói chung việc truyền tham số không dễ sai, nên mục này không có tác dụng lớn lắm.

4 Quy trình gỡ lỗi tổng quát

Với một bài toán, ta thường gỡ lỗi theo quy trình sau. Flowchart trong bản gốc có thể diễn đạt bằng các bước:

1. Bắt đầu gỡ lỗi.
2. Tìm lỗi tĩnh. Nếu chưa qua, sửa lỗi và mã hóa lại rồi quay lại.
3. Kiểm thử tổng thể. Nếu qua, chuyển sang kiểm tra ca kiểm thử tiếp theo.
4. Nếu kiểm thử tổng thể chưa qua, kiểm thử mô-đun. Nếu mô-đun chưa qua, theo dõi gỡ lỗi, sửa lỗi và mã hóa lại.
5. Nếu mô-đun đã qua, kiểm tra xem mọi mô-đun đã được kiểm thử xong chưa. Nếu chưa, tiếp tục kiểm thử mô-đun; nếu rồi, quay lại sửa lỗi và mã hóa lại khi cần.

6. Khi một ca kiểm thử đã qua, kiểm tra còn ca kiểm thử nào chưa. Nếu còn, tiếp tục kiểm thử; nếu hết, kết thúc gỡ lỗi.

Đây chỉ là trường hợp thông thường. Khi gỡ lỗi cụ thể, có thể thay đổi một số bước; chẳng hạn sau khi phát hiện một mô-đun có lỗi và sửa xong, có thể không quay lại giai đoạn tìm lỗi tĩnh mà tiếp tục tìm lỗi trong mô-đun ấy. Điều này tùy tình hình cụ thể.

Dưới đây ta trình bày chi tiết từng bước.

4.1 Tìm lỗi tĩnh

Tìm lỗi tĩnh là không thực thi chương trình, mà chỉ dựa vào chương trình và mô tả quá trình giải trong sơ đồ để phát hiện lỗi. Vì có những lỗi rất khó tìm khi chạy, nhưng lại dễ phát hiện trong tìm lỗi tĩnh, chẳng hạn lỗi viết nhầm đã nói ở trên, nên tìm lỗi tĩnh thường là bước kiểm tra không thể xem nhẹ. Thứ tự tìm lỗi tĩnh thường là kiểm tra cục bộ trước, rồi kiểm tra toàn cục. Kiểm tra cục bộ chủ yếu là đọc lập kiểm tra chức năng của từng mô-đun con có được cài đặt theo yêu cầu thuật toán hay không; kiểm tra toàn cục chủ yếu là xem giao diện giữa các phần cục bộ có khớp không, có thiếu chức năng nào không. Trong giai đoạn tìm lỗi tĩnh, ta có thể kiểm tra có mục tiêu các loại lỗi đã liệt kê ở trên. Càng tìm ra nhiều lỗi trong giai đoạn này, thời gian gỡ lỗi càng được tiết kiệm.

4.2 Tìm lỗi động

Tìm lỗi tĩnh có thể phát hiện lỗi, nhưng không thể bảo đảm không còn lỗi, vì ở đây có yếu tố con người: chỉ cần lơ đãng là có thể bỏ sót một lỗi. Vì vậy, ta cần kết hợp với tìm lỗi động để tìm những lỗi bị bỏ sót. Đối lập với tìm lỗi tĩnh, tìm lỗi động là phát hiện lỗi bằng cách theo dõi quá trình thực thi của chương trình và đối chiếu kết quả đầu ra. Kỹ thuật tìm lỗi động có thể chia thành hai phần lớn: thiết kế ca kiểm thử và phương pháp kiểm thử. Trước hết nói về phương pháp kiểm thử.

Phương pháp kiểm thử trong tìm lỗi động có thể phân loại theo hai tiêu chuẩn.

4.2.1 Theo căn cứ thiết kế ca kiểm thử

Theo căn cứ thiết kế ca kiểm thử khác nhau, có thể chia thành kiểm thử hộp đen và kiểm thử hộp trắng.

Phương pháp chỉ biết chức năng nhập và xuất của chương trình, không biết cấu trúc cụ thể của chương trình, rồi thiết kế ca kiểm thử bằng cách liệt kê các tình huống có thể khác nhau và chạy chúng để phát hiện lỗi được gọi là kiểm thử hộp đen. Phương pháp biết cấu trúc và luồng bên trong chương trình, dựa trên logic nội bộ của chương trình để thiết kế ca kiểm thử và chạy chúng để phát hiện lỗi được gọi là kiểm thử hộp trắng. Trong thi đấu, ta thường kết hợp cả hai. Khi kiểm thử mô-đun tầng dưới có thể dùng kiểm thử hộp trắng, dựa vào điều kiện và dữ liệu kiểm thử chuyên biệt để xét trạng thái của chương trình tại các điểm khác nhau có đúng như mong đợi không. Khi gỡ lỗi tổng thể, có thể dùng kiểm thử hộp đen, tách khỏi cấu trúc bên trong chương trình để xem chương trình có giải đúng với dữ liệu kiểm thử trong các tình huống khác nhau không. Với mô-đun trung gian, có thể dùng hộp đen, cũng có thể dùng hộp trắng, hoặc dùng cả hai, tùy phương pháp nào thích hợp. Nói chung, mô-đun có cấu trúc phức tạp dùng kiểm thử hộp đen, mô-đun có cấu trúc đơn giản dùng kiểm thử hộp trắng. Cuối cùng, vì ca kiểm thử hộp trắng khó thiết kế hơn, và các kỳ thi tin học hiện nay đều chấm bằng kiểm thử hộp đen, nên trong điều kiện thời gian thi đấu gấp, có thể thống nhất dùng kiểm thử hộp đen để đạt hiệu suất cao hơn.

4.2.2 Theo thứ tự kiểm thử

Theo thứ tự kiểm thử khác nhau, có thể chia thành kiểm thử từ dưới lên và kiểm thử từ trên xuống.

Kiểm thử từ dưới lên là kiểm thử mô-đun tầng thấp nhất trước, rồi lần lượt kiểm thử lên trên, cuối cùng kiểm thử mô-đun chính. Kiểm thử từ trên xuống ngược lại: kiểm thử mô-đun chính trước, rồi theo thứ tự thiết kế từ trên xuống mà lần lượt kiểm thử các mô-đun. Để tăng tốc độ gỡ lỗi, nên dùng thứ tự kiểm thử từ trên xuống; chỉ khi phát hiện mô-đun nào đó có lỗi mới đi vào tầng dưới để gỡ, cách này cũng rất có ích cho việc nhanh chóng định vị lỗi.

Khi vận dụng các phương pháp kiểm thử này, cần chú ý điều gì? Trước hết, phải nắm rất rõ cấu trúc chương trình mình viết và chức năng của từng mô-đun. Khi dùng phương pháp kiểm thử từ trên xuống, thường một mô-đun chưa kiểm thử xong đã chuyển sang mô-đun tiếp theo, nên đặc biệt dễ quên và bỏ sót. Nếu trong đầu không có khái niệm về cấu trúc chương trình, ta rất dễ bị rối. Nếu không rõ chức năng của mô-đun, cũng khó phán đoán kết quả thực thi của mô-đun đúng hay sai, từ đó không thể xác định chính xác nơi có lỗi. Thứ hai, việc kiểm thử cần có trật tự. Hãy kiên trì dùng cùng một ca kiểm thử cho tới khi đầu ra đúng; khi một mô-đun chưa kiểm thử xong thì không kiểm thử mô-đun kế tiếp, trừ khi mô-đun ấy là mô-đun con của mô-đun hiện tại và trong lúc kiểm thử mô-đun hiện tại phát hiện mô-đun con này có lỗi. Cuối cùng, sau mỗi lần sửa mã nguồn, nhất định phải chạy lại toàn bộ các ca kiểm thử đã chạy để phòng sinh ra lỗi mới.

Sau phương pháp kiểm thử, ta nói về thiết kế ca kiểm thử.

Ca kiểm thử của phương pháp hộp đen được thiết kế theo các tình huống khác nhau của dữ liệu nhập. Vì dữ liệu nhập của một bài có thể muôn hình vạn trạng, kiểm thử hộp đen không thể kiểm hết mọi tình huống. Chẳng hạn có một bài yêu cầu nhập hai số nguyên kiểu LongInt là X, Y , rồi in ra tổng của chúng. X có 2^{32} giá trị khác nhau, Y cũng có 2^{32} giá trị khác nhau, do đó dữ liệu nhập khác nhau có $2^{32} \cdot 2^{32} = 2^{64}$ tình huống. Giả sử chạy chương trình một lần mất 1 mili giây, muốn kiểm hết mọi tình huống khác nhau cần khoảng năm trăm triệu năm, hiển nhiên không thể làm được.

Ca kiểm thử của phương pháp hộp trắng được thiết kế theo logic nội bộ của chương trình. Muốn kiểm thử toàn bộ chương trình hoàn toàn, ca kiểm thử phải phủ mọi đường thực thi của chương trình, điều này thường cũng không thể. Ví dụ, nếu sơ đồ luồng của một chương trình có năm đường đi khác nhau từ điểm a tới điểm b , rồi bọc ngoài bằng vòng lặp 20 lần, theo nguyên lý nhân, số đường khác nhau mà một lần chạy chương trình có thể đi qua là

$$5^{20} \approx 10^{14}.$$

Nếu chương trình chạy từ a tới b mất 0,1 giây, kiểm thử hết mọi đường đi sẽ cần hơn một triệu năm, rõ ràng cũng không thể.

Từ hai ví dụ trên có thể thấy, tìm lỗi động cũng như tìm lỗi tĩnh: chúng chỉ có thể phát hiện sự tồn tại của một số lỗi, chứ không thể chứng minh lỗi không tồn tại. Vì vậy, khi thiết kế ca kiểm thử, ta cần xét tính kinh tế của ca kiểm thử.

“Tính kinh tế” nghĩa là dùng càng ít ca kiểm thử càng tốt để phủ càng nhiều tình huống càng tốt. Với kiểm thử hộp đen, đó là bao gồm càng nhiều loại dữ liệu nhập khác nhau càng tốt; với kiểm thử hộp trắng, đó là phủ càng nhiều đường thực thi càng tốt. Xét rằng trong thi đấu kiểm thử chủ yếu là hộp đen, ta chỉ bàn thiết kế ca kiểm thử hộp đen. Các phương pháp thiết kế thường dùng gồm phân lớp tương đương, phân tích biên và dự đoán lỗi.

4.2.3 Phân lớp tương đương

Phân lớp tương đương là dựa vào chức năng của chương trình để chia dữ liệu nhập thành một số lớp tương đương, rồi xét việc chọn dữ liệu và thiết kế ca kiểm thử nhằm đạt mục đích kiểm thử.

Ví dụ có bài: nhập ba số nguyên biểu thị độ dài ba cạnh của một tam giác. Yêu cầu in ra chúng có tạo được tam giác hay không; nếu được thì in ra đó là tam giác cân, tam giác đều, hay không đều cũng không cân.

Với bài này, dùng phân lớp tương đương, ta có thể chia dữ liệu nhập thành hai lớp theo việc tam giác có hợp lệ hay không: tam giác hợp lệ và tam giác không hợp lệ. Với tam giác hợp lệ, có thể tiếp tục chia thành tam giác cân và tam giác vừa không đều vừa không cân; với tam giác cân, còn có thể chia thành tam giác cân thuận túy và tam giác đều. Bằng cách liên tục phân lớp tương đương, cuối cùng ta có thể thiết kế bốn loại ca kiểm thử. Ngoài ra, cũng có thể phân lớp theo các cách sắp xếp dữ liệu nhập khác nhau. Nhưng nếu chương trình của bạn trước hết sắp xếp rồi mới phán đoán, và có thể bảo đảm phần sắp xếp đúng, thì không cần dùng cách phân lớp này.

4.2.4 Phân tích biên

Chương trình thường chạy sai ở trạng thái biên, nên trong kiểm thử ta thường trước hết căn cứ vào chức năng chương trình để xác định sự thay đổi dữ liệu ở tình huống biên, từ đó thiết kế ca kiểm thử. Phương pháp phân tích trạng thái biên để thiết kế ca kiểm thử chương trình được gọi là phân tích biên.

Việc chọn giá trị biên cần có mục tiêu và một mức sáng tạo nhất định theo tình hình thực tế của đề. Nói chung, có thể xét các tình huống sau:

1. dữ liệu vừa ở gần biên và sắp vượt biên;
2. dữ liệu lấy giá trị lớn nhất hoặc nhỏ nhất, nhiều nhất hoặc ít nhất cộng trừ 1;
3. dữ liệu là giá trị đầu và cuối của vòng lặp hoặc phép lặp;
4. phần tử dữ liệu đầu tiên hoặc cuối cùng của tập có thứ tự;
5. phần tử gần điểm không;
6. dữ liệu ở sai số lớn nhất;
7. dữ liệu có lượng dữ liệu lớn nhất hoặc nhỏ nhất;
8. dữ liệu có lượng tính toán lớn nhất hoặc nhỏ nhất.

Vẫn dùng bài tam giác ở trên làm ví dụ, có ba dữ liệu kiểm thử sau:

1. tổng hai cạnh đúng bằng cạnh thứ ba;
2. trong ba số có chứa 0;
3. cả ba số đều bằng 0.

Trường hợp thứ nhất thuộc loại dữ liệu gần biên; hai trường hợp sau thuộc loại phần tử gần điểm không. Đó là ứng dụng của phương pháp phân tích biên.

Phân tích biên là một phương pháp rất quan trọng, không thể thiếu trong kiểm thử nói chung. Nó tinh gọn và dễ phát hiện lỗi. Vấn đề là tình huống biên của một số chương trình cực kỳ phức tạp, thậm chí đôi khi không tồn tại. Ví dụ, nếu yêu cầu sắp xếp 10 số theo thứ tự giảm dần rồi in ra, vì thuật toán chỉ liên quan đến quan hệ lớn nhỏ giữa hai số, không liên quan đến giá trị cụ thể của từng số, còn tổng số phần tử lại cố định, lúc này không có tình huống biên, nên không thể dùng phân tích biên để thiết kế ca kiểm thử.

4.2.5 Dự đoán lỗi

Phương pháp dự đoán lỗi thực chất tận dụng tư tưởng kết hợp hộp đen và hộp trắng: dựa vào chương trình do chính mình thiết kế để tìm ra những nơi dễ phát sinh lỗi, rồi thiết kế ca kiểm thử có mục tiêu. Chẳng hạn ở một nơi có nhiều câu If-Then-Else lồng nhau, lỗi rất dễ xuất hiện, mà ca kiểm thử thông thường lại khó tìm ra. Khi đó dự đoán lỗi phát huy tác dụng lớn: với chuỗi câu If-Else này, thiết kế ca kiểm thử phủ các đường đi khác nhau để kiểm nghiệm tính đúng đắn của chương trình.

Cụ thể khi kiểm thử nên chọn phương pháp nào? Chiến lược thường dùng là trước hết dùng phân tích biên, rồi dùng phân lớp tương đương để bổ sung. Cuối cùng, với những phần có cấu trúc phức tạp trong chương trình, trọng điểm dùng phương pháp dự đoán lỗi để kiểm thử. Tất nhiên, nếu thời gian cho phép, nên dùng nhiều dữ liệu kiểm thử hơn để phủ nhiều chức năng hơn.

4.3 Theo dõi gỡ lỗi

Theo dõi gỡ lỗi thường là việc rườm rà, đòi hỏi sự kiên nhẫn và cẩn thận của thí sinh. Chọn biến nào để theo dõi cũng cực kỳ quan trọng; chọn biến chính xác có thể đạt hiệu quả gấp đôi với nửa công sức. Nói chung, trước hết cần theo dõi các biến lưu dữ liệu nhập, đặc biệt với những bài cần xử lý dữ liệu nhập trước thì càng phải làm vậy. Nếu dữ liệu nhập không đúng, ngay cả chương trình đúng cũng sẽ không khớp đáp án. Tiếp theo là các biến toàn cục thường xuyên được dùng; những biến này thường xuyên suốt cả chương trình, một khi sai ở đâu đó sẽ ảnh hưởng tính đúng đắn của các mô-đun khác, khiến việc định vị lỗi bị lệch: nơi có lỗi lại không được kiểm thử, nơi đúng lại bị kiểm thử lặp đi lặp lại. Vì vậy, phải chú ý chặt chẽ sự thay đổi của các biến toàn cục này, không được bỏ qua bất kỳ lỗi nào. Kế đến là các biến vòng lặp; theo dõi biến vòng lặp giúp biết chính xác tiến độ thực thi của chương trình, từ đó nắm được lỗi theo thời gian. Cuối cùng, các biến khác cần chọn theo tình hình thực tế; biến nào được dùng nhiều hơn thì nên ưu tiên theo dõi.

Ngoài ra, kích thước và vị trí cửa sổ Watch cũng cần được sắp xếp hợp lý, sao cho trong quá trình theo dõi, sự thay đổi của các biến quan trọng có thể thấy rõ ngay, tránh nhiều lần chuyển cửa sổ không cần thiết và tiết kiệm thời gian.

5 Tổng kết

Trong thi tin học, hiệu suất chương trình cao hay thấp không phải là điều quan trọng nhất. Chương trình không đúng, dù hiệu suất cao đến đâu, cũng bằng không. Chương trình hiệu suất thấp nhưng đúng thì luôn có thể giành một phần điểm. Vì vậy, trình độ gỡ lỗi ở mức độ lớn cũng phản ánh trình độ tổng thể của thí sinh. Nói chung, gỡ lỗi chương trình chủ yếu vẫn dựa vào kinh nghiệm. Kinh nghiệm càng phong phú, hiệu quả gỡ lỗi càng tốt. Do đó, trong luyện tập hằng ngày, ta tuyệt đối không nên chỉ chú ý rèn luyện ý tưởng và thuật toán, mà còn phải chú ý rèn luyện trình độ gỡ lỗi của mình.

6 Một ví dụ gỡ lỗi

Đề bài

Tại một số thành phố lớn ở Hoa Kỳ, dịch vụ chuyển phát bằng xe đạp để gửi tài liệu và vật nhỏ cho doanh nghiệp từ lâu đã là một phần của dịch vụ vận chuyển lưu động. Những người đi xe ở Boston là một nhóm khác thường. Họ nổi tiếng vì phóng quá tốc độ, không tuân thủ đường một chiều và đèn giao thông, phớt lờ sự tồn tại của ô tô, taxi, xe buýt và người đi bộ. Cạnh tranh trong dịch vụ chuyển phát rất gay gắt. Công ty Billy's Bicycle Messenger Service (BBMS) cũng không ngoại lệ. Để phát triển kinh doanh và đặt mức phí hợp lý, BBMS đang xây dựng tiêu chuẩn tính phí dựa trên tuyến đường ngắn nhất mà nhân viên chuyển phát có thể đi. Nhiệm vụ của bạn là viết chương trình cho BBMS để xác định độ dài các tuyến đường ấy.

Các giả thiết sau giúp đơn giản hóa công việc:

- Nhân viên chuyển phát có thể đi xe ở mọi nơi trên mặt đất, trừ bên trong các tòa nhà.

- Tòa nhà có địa hình không đều có thể coi là hợp của một số hình chữ nhật. Quy định rằng mọi hình chữ nhật giao nhau có phần trong chung và là một phần của cùng một tòa nhà.
- Dù hai tòa nhà khác nhau có thể rất gần nhau, chúng không bao giờ chồng lấn. Nhân viên chuyển phát có thể đi qua giữa hai tòa nhà bất kỳ; họ có thể vòng qua các góc gắt nhất và đi sát mép tòa nhà.
- Điểm xuất phát và điểm kết thúc không nằm bên trong tòa nhà.
- Luôn tồn tại một tuyến nối điểm đầu và điểm cuối.

Bản gốc kèm một sơ đồ nhìn từ trên xuống: trong khung tọa độ từ 0 đến 12, điểm Start nằm phía trên bên trái, điểm Stop nằm gần giữa phía dưới, và vài tòa nhà dạng hình chữ nhật nghiêng nằm giữa đường đi.

Tập nhập gồm các dòng sau:

Dòng đầu: n , số hình chữ nhật mô tả các tòa nhà trong cảnh, $0 \leq n \leq 20$.

Dòng thứ hai: x_1, y_1, x_2, y_2 , tọa độ x, y của điểm đầu và điểm cuối tuyến đường.

n dòng còn lại: $x_1, y_1, x_2, y_2, x_3, y_3$, tọa độ ba đỉnh của một hình chữ nhật.

Mọi tọa độ x, y là số thực trong đoạn từ 0 đến 1000, kể cả hai đầu mút. Các tọa độ kề nhau trên cùng một dòng cách nhau bởi một hoặc nhiều dấu cách. Như ví dụ đầu ra dưới đây, đầu ra cần cho độ dài tuyến ngắn nhất từ điểm đầu đến điểm cuối, chính xác tới hai chữ số sau dấu thập phân.

Ví dụ nhập.

```
5
6.5 9 10 3
1 5 3 3 6 6
5.25 2 8 2 8 3.5
6 10 6 12 9 12
7 6 11 6 11 8
10 7 11 7 11 11
```

Ví dụ xuất.

```
7.28
```

Phân tích thuật toán

Từ định dạng tập nhập có thể thấy, mỗi hình chữ nhật được nhập bằng ba đỉnh P_1, P_2, P_3 . Tọa độ ba đỉnh lần lượt là $(P_1.x, P_1.y)$, $(P_2.x, P_2.y)$, $(P_3.x, P_3.y)$. Vì bốn cạnh hình chữ nhật không nhất thiết song song với trục x và y , ta trước hết sắp xếp P_1, P_2, P_3 sao cho P_2 ở bên phải hoặc ngay phía trên P_1 , tức là

$$(P_2.x > P_1.x) \text{ hoặc } (P_2.x = P_1.x \text{ và } P_2.y \geq P_1.y),$$

P_3 ở bên phải hoặc ngay phía trên P_1 , và P_3 ở bên phải hoặc ngay phía trên P_2 . Sau đó tính đỉnh thứ tư của hình chữ nhật:

$$P_4.x = P_1.x + P_3.x - P_2.x, \quad P_4.y = P_1.y + P_3.y - P_2.y.$$

Ví dụ, ba đỉnh nhập vào của một hình chữ nhật là $(3, 3)$, $(1, 5)$, $(6, 6)$. Sau khi sắp xếp theo cách trên:

$$P_1 = (1, 5), \quad P_2 = (3, 3), \quad P_3 = (6, 6).$$

Khi đó

$$P_4.x = 1 + 6 - 3 = 4, \quad P_4.y = 5 + 6 - 3 = 8,$$

tức $P_4 = (4, 8)$.

Từ định nghĩa sắp xếp có thể thấy, với bốn đỉnh của mỗi hình chữ nhật, P_1 và P_2 là một cặp đỉnh đối nhau, P_3 và P_4 là cặp đỉnh đối nhau còn lại.

Vì nhân viên chuyển phát tìm đường ngắn nhất trong không gian trên mặt đất trừ phần trong của tòa nhà, nhưng được đi trên mép tòa nhà, nên các đỉnh trên tuyến đường này, ngoài điểm đầu và điểm cuối, đều là đỉnh của các hình chữ nhật. Hai đầu mút của mỗi đoạn thẳng trên đường ngắn nhất không được bị bất kỳ tòa nhà nào chắn, và cũng không được là hai đỉnh đối nhau của cùng một hình chữ nhật, vì như vậy sẽ đi xuyên qua tòa nhà tương ứng.

Ta đưa điểm đầu, điểm cuối và mọi đỉnh hình chữ nhật vào một bảng đỉnh P , trong đó $P[1]$ và $P[4n + 2]$ lưu điểm đầu và điểm cuối, $P[2] \dots P[4n + 1]$ lưu các đỉnh của n hình chữ nhật. Rõ ràng, các đỉnh trên đường ngắn nhất lấy từ bảng P . Trong khi xây dựng bảng P , đồng thời xây dựng bảng đoạn thẳng L , lưu bốn cạnh và hai đường chéo của mỗi hình chữ nhật vào bảng này. Mục đích đặt bảng L là để phán đoán tuyến đi của nhân viên chuyển phát có hợp lệ hay không. Nếu tuyến đi giao với bất kỳ cạnh hình chữ nhật nào trong bảng, nghĩa là nhân viên đã đi vào bên trong tòa nhà, trường hợp này phải loại bỏ. Vấn đề là: tại sao bảng L còn cần lưu hai đường chéo của mỗi hình chữ nhật? Đó là để loại bỏ một lỗi biên khó thấy: nếu điểm đầu và điểm cuối cùng nằm trên cạnh của một hình chữ nhật, đây là tình huống được phép vì có thể đi sát mép tòa nhà, thì chương trình có thể vô tư cho đi xuyên qua vùng cấm, vì đoạn nối hai điểm không giao với cạnh hình chữ nhật nào. Thêm điều kiện tuyến đi không được giao với đường chéo của hình chữ nhật sẽ tránh được lỗi hổng khó phát hiện này.

Việc tìm đường ngắn nhất có thể dùng phương pháp gán nhãn.

Chương trình

```
Program Corner;
Const
  FinName='corner.in';{ten tep nhap}
  FoutName='corner.out';{ten tep xuat}
  MaxBuilding=20;{so toa nha toi da}
  MaxPoint=82;{so dinh toi da}
  MaxLine=120;{so doan thang toi da}
  Zero=1e-8;{luong rat nho tuong doi, dung de so sanh so thuc}
Type
  Location=Record{kieu toa do}
    x,y:Real;
  End;
  Line=Array[1..2]Of Location;{kieu doan thang}
Var
  Bld,{so toa nha}
  Pnt,{so dinh}
  Lne:Integer;{so doan thang}
  P:Array[1..MaxPoint]Of Location;{day dinh}
  L:Array[1..MaxLine]Of Line;{day doan thang}
  Dis:Array[1..MaxPoint]Of Real;{bang khoang cach ngan nhât}
Function GetMin(Var a,b:Real):Real;{tra ve so nho hon}
```

```

Begin
  If a>b Then GetMin:=b Else GetMin:=a;
End;
Function GetMax(Var a,b:Real):Real;{tra ve so lon hon}
Begin
  If a>b Then GetMax:=a Else GetMax:=b;
End;
Function GetMulti(p0,p1,p2:Location):Real;{tinh tich co huong}
Begin
  GetMulti:=(p1.x-p0.x)*(p2.y-p0.y)-(p2.x-p0.x)*(p1.y-p0.y);
End;
Function Intersect(Var L1,L2:Line):Boolean;{xet hai doan thang co giao nhau}
Var
  M1,M2,M3,M4:Real;
Begin
  Intersect:=False;
  If (GetMin(L1[1].x,L1[2].x)<=GetMax(L2[1].x,L2[2].x))
    And(GetMax(L1[1].x,L1[2].x)>=GetMin(L2[1].x,L2[2].x))
    And(GetMin(L1[1].y,L1[2].y)<=GetMax(L2[1].y,L2[2].y))
    And(GetMax(L1[1].y,L1[2].y)>=GetMin(L2[1].y,L2[2].y)) Then
    {phep loai tru nhanh}
    Begin
      M1:=GetMulti(L1[1],L2[1],L1[2]);M2:=GetMulti(L1[1],L1[2],L2[2]);
      M3:=GetMulti(L2[1],L1[1],L2[2]);M4:=GetMulti(L2[1],L2[2],L1[2]);
      If (Abs(M1)>Zero)And(Abs(M2)>Zero)
        And(Abs(M3)>Zero)And(Abs(M4)>Zero)
        And(M1*M2>0)And(M3*M4>0) Then Intersect:=True;
    End;
  End;
End;
Procedure GetNextPoint(Var P1,P2,P3,P4:Location);
{sap xep dinh va tinh toa do dinh thu tu}
Var
  T:Location;
Begin
  If (P2.x<P1.x)Or((P2.x=P1.x)And(P2.y<P1.y)) Then
    Begin T:=P1;P1:=P2;P2:=T;End;
  If (P3.x<P2.x)Or((P3.x=P2.x)And(P3.y<P2.y)) Then
    Begin T:=P3;P3:=P2;P2:=T;End;
  P4.x:=P1.x+P3.x-P2.x;
  P4.y:=P1.y+P3.y-P2.y;
End;
Procedure Init;{doc du lieu va khoi tao}
Var
  i,j:Byte;
Begin
  ReadLn(Bld);
  Lne:=0;Pnt:=1;
  ReadLn(P[1].x,P[1].y,P[Bld*4+2].x,P[Bld*4+2].y);
  For i:=1 to Bld Do
    Begin
      For j:=1 to 3 Do
        Read(P[Pnt+j].x,P[Pnt+j].y);
      ReadLn;
      GetNextPoint(P[Pnt+1],P[Pnt+2],P[Pnt+3],P[Pnt+4]);
    End;
  End;

```

```

For j:=1 to 3 Do
  Begin
    L[Lne+j,1]:=P[Pnt+j];
    L[Lne+j,2]:=P[Pnt+j+1];
  End;
  L[Lne+4,1]:=P[Pnt+4];L[Lne+4,2]:=P[Pnt+1];
  L[Lne+5,1]:=P[Pnt+1];L[Lne+5,2]:=P[Pnt+3];
  L[Lne+6,1]:=P[Pnt+2];L[Lne+6,2]:=P[Pnt+4];
  Inc(Pnt,4);Inc(Pnt,6);
End;
Inc(Pnt);
End;
Function GetDis(a,b:Byte):Real;{tinh khoang cach giua hai diem}
Begin
  GetDis:=Sqrt((P[a].x-P[b].x)*(P[a].x-P[b].x)
    +(P[a].y-P[b].y)*(P[a].y-P[b].y));
End;
Function Visible(Var P1,P2:Location):Boolean;
{xet tu P1 co nhin thay P2 khong, tuc giua hai diem co bi toa nha chan khong}
Var
  TL:Line;
  i:Byte;
Begin
  TL[1]:=P1;TL[2]:=P2;
  For i:=1 to Lne Do
    If Intersect(L[i],TL) Then
      Begin Visible:=False;Exit;End;
  Visible:=True;
End;
Procedure Main;{chuong trinh chinh, dung gan nhan de tim duong ngan nhât}
Var
  i,j,k:Byte;
  Min:Real;
  S:Set Of Byte;
Begin
  Dis[1]:=0;
  For i:=2 to Pnt Do
    If Visible(P[1],P[i]) Then Dis[i]:=GetDis(1,i)
    Else Dis[i]:=1e10;
  S:=[];
  For i:=2 to Pnt Do
    Begin
      Min:=1e10;k:=0;
      For j:=2 to Pnt Do
        If (Not(j In S))And(Dis[j]<Min) Then Begin k:=j;Min:=Dis[j];End;
      If k=Pnt Then Break;
      S:=S+[k];
      For j:=2 to Pnt Do
        If (Not(j In S))And(Visible(P[k],P[j]))
          And(Dis[k]+GetDis(j,k)<Dis[j]) Then
          Dis[j]:=Dis[k]+GetDis(j,k);
      End;
    End;
  WriteLn(Dis[Pnt]:0:2);{xuat dap an}
End;

```

```

Begin
  Assign(Input,FinName);
  Reset(Input);
  Assign(Output,FoutName);
  Rewrite(Output);
  Init;
  Main;
  Close(Input);
  Close(Output);
End.

```

Chương trình này đã biên dịch qua. Bây giờ ta tiến hành tìm lỗi tĩnh. Trước hết xác định các hàm GetMin, GetMax, GetMulti là đúng. Vì ba hàm này rất ngắn và rất đơn giản, chỉ cần tìm lỗi tĩnh là được. Xem tiếp xuống dưới, trong thủ tục Init có một dòng `Inc(Pnt,4); Inc(Pnt,6);` rõ ràng không đúng. Ý định của chương trình rõ ràng là tăng giá trị của Pnt và Lne, nên phải thay `Inc(Pnt,6)` bằng `Inc(Lne,6)`. Xem tiếp, trong hàm Intersect, khi tính tích có hướng, chương trình gán giá trị cho M2 hai lần liên tiếp, làm lần gán trước không còn ý nghĩa; đây rõ ràng là lỗi viết nhầm, M2 phía sau cần sửa thành M4. Xem tiếp nữa thì không phát hiện lỗi khác.

Tiếp theo vào tìm lỗi động. Ví dụ đầu tiên dùng đương nhiên là ví dụ mẫu. Sau khi nhập mẫu, chương trình in ra 7.28, khớp đáp án. Vì vậy không cần kiểm thử mô-đun. Bây giờ cần tự thiết kế ca kiểm thử.

Trước hết dùng phân tích giá trị biên. Bài này có hai biên ở số lượng tòa nhà: 0 và 20. Ca kiểm thử biên trên khá rườm rà, nên trước hết thử biên dưới 0 và lân cận biên dưới là 1. Tọa độ cũng có hai biên: 0 và 1000. Kết hợp biên số tòa nhà, ta nhận được hai ca kiểm thử sau:

```

corner1.in
0
0 0 1000 1000

```

```

corner2.in
1
0 0 1000 1000
0 0 0 1000 1000 1000

```

Với corner1, chương trình in 1414.21, đúng. Với corner2, chương trình in 2000.00, cũng đúng. Tiếp theo ta dùng phân lớp tương đương để thiết kế ca kiểm thử.

Trước hết phân lớp theo cách sắp xếp dữ liệu. Ta chỉ dùng một hình chữ nhật, rồi thử cả sáu cách hoán vị ba đỉnh của nó; như vậy có sáu ca kiểm thử. Dưới đây chỉ liệt kê một ca:

```

corner3.in
1
0 0 1 1
0 0 0 1 1 1

```

Đáp án phải đều là 2.00, nhưng khi đo tới cách sắp xếp đỉnh (0,1), (1,1), (0,0), chương trình in 1.41. Vì các điều kiện khác đều không đổi, khả năng lớn nhất là thủ tục GetNextPoint sai. Dùng Go to cursor chạy tới đầu GetNextPoint, thêm mảng biến P vào cửa sổ Watch, rồi dùng Trace into chạy từng bước. Ta thấy chương trình không hoán đổi P_1, P_2 , có hoán đổi P_2, P_3 ; nhưng theo thuật toán, rõ ràng cần đặt (0,0) vào vị trí đầu tiên. Từ đó vỡ lẽ rằng sau khi so sánh P_1, P_2 , cần so sánh tiếp P_1, P_3 , cuối cùng mới so sánh P_2, P_3 . Vì vậy, sau câu If-Then-Else đầu tiên cần thêm:

```
If (P3.x<P1.x)Or((P3.x=P1.x)And(P3.y<P1.y)) Then
Begin T:=P1;P1:=P3;P3:=T;End;
```

Kiểm tra lại, đáp án không sai, các ca kiểm thử trước cũng đều qua. Ta tiếp tục kiểm thử tiếp.

Lần này phân lớp theo quan hệ giữa các hình chữ nhật: hai hình chữ nhật có thể chứa nhau, giao nhau, tiếp xúc hoặc rời nhau. Ở đây “tiếp xúc” nghĩa là hai hình chữ nhật có một phần cạnh trùng nhau nhưng không chứa nhau. Vì đề quy định hai tòa nhà khác nhau có thể rất gần nhưng không bao giờ chồng lấn, ta loại trừ tình huống tiếp xúc, nhưng có thể cho hai hình chữ nhật rời nhau đứng rất gần. Lần này dùng hai hình chữ nhật, có ba ca kiểm thử sau:

```
corner9.in {chua nhau}
```

```
2
1 0 1 3
0 0 0 3 3 3
1 1 1 2 2 2
```

```
corner10.in {giao nhau}
```

```
2
1 0 1 1
0 0 0 1 2 1
1 0 1 1 3 1
```

```
corner11.in {roi nhau nhưng rat gan}
```

```
2
1 0 1 1
0 0 0 1 1 1
1.0001 0 1.0001 1 3 1
```

Hai ca 10 và 11 không có vấn đề; ca thứ 9 in ra 3, có vấn đề, đáp án đáng ra phải là 5. Ta vào gỡ lỗi mô-dun. Dữ liệu đọc vào và khởi tạo không có vấn đề. Tiếp tục đi vào Main để theo dõi; trước hết dùng Go to cursor chạy xong vòng lặp đầu tiên trong một bước, kiểm tra mảng Dis, phát hiện ngoài điểm đầu còn có bốn điểm có giá trị nhỏ hơn 10^{10} . Vì đây là ca một hình chữ nhật chứa một hình chữ nhật khác, theo lý thì chỉ có hai điểm có giá trị nhỏ hơn 10^{10} . Kiểm tra bốn điểm ấy, phát hiện hình chữ nhật bị chứa có hai điểm cũng được tính vào. Suy nghĩ kỹ thì hiểu ra: tuy ta đã thêm hai đường chéo, với những điểm nằm bên trong hình chữ nhật thì vẫn không thể loại bỏ. Đường ngắn nhất mà chương trình chọn chắc là đi theo đường thẳng; vì giữa đường có thêm hai đỉnh của hình chữ nhật chia nó thành ba đoạn, hai đỉnh này lại vừa đúng nằm trên đường chéo, nên cả ba đoạn đều không giao với đường chéo. Chương trình vì thế chọn đi theo “đường ngắn nhất” ấy. Cách giải quyết là: vì điểm đầu và điểm cuối đều không nằm trong hình chữ nhật, sau khi đọc xong mọi đỉnh, ta có thể loại bỏ những điểm nằm bên trong hình chữ nhật. Phần sửa cụ thể xem phụ lục.

Ta còn có thể phân lớp theo quan hệ vị trí giữa điểm đầu và điểm cuối. Vì điều này liên quan tới phân bố hình chữ nhật và tình huống khá phức tạp, ta chỉ xét điểm đầu và điểm cuối có trùng nhau hay không, nên thêm một ca kiểm thử:

```
corner11.in
```

```
0
0 0 0 0
```

Đầu ra là 0.00, đúng.

Cuối cùng dùng phương pháp dự đoán lỗi. Trong bài này, phức tạp nhất chắc chắn là hai hàm Intersect và InBuilding. Vì vậy có thể nhấm riêng vào hai hàm này, coi chúng lần lượt như một bài nhỏ để kiểm thử. Có thể tham khảo các bước ở trên để thiết kế ca kiểm thử: trước dùng phân tích giá trị biên, sau dùng phân lớp tương đương. Ca kiểm thử cụ thể không liệt kê nữa. Kết quả sau khi kiểm thử như vậy cũng không có lỗi. Việc kiểm thử toàn bài tới đây hoàn tất.

Phụ lục

Chương trình sau khi sửa

```

Program Corner;
Const
  FinName='corner.in';{ten tep nhap}
  FoutName='corner.out';{ten tep xuat}
  MaxBuilding=20;{so toa nha toi da}
  MaxPoint=82;{so dinh toi da}
  MaxLine=120;{so doan thang toi da}
  Zero=1e-8;{luong rat nho tuong doi, dung de so sanh so thuc}
Type
  Location=Record{kieu toa do}
    x,y:Real;
  End;
  Line=Array[1..2]Of Location;{kieu doan thang}
Var
  Bld,{so toa nha}
  Pnt,{so dinh}
  Lne:Integer;{so doan thang}
  P:Array[1..MaxPoint]Of Location;{day dinh}
  L:Array[1..MaxLine]Of Line;{day doan thang}
  Dis:Array[1..MaxPoint]Of Real;{bang khoang cach ngan nhat}
Function GetMin(Var a,b:Real):Real;{tra ve so nho hon}
Begin
  If a>b Then GetMin:=b Else GetMin:=a;
End;
Function GetMax(Var a,b:Real):Real;{tra ve so lon hon}
Begin
  If a>b Then GetMax:=a Else GetMax:=b;
End;
Function GetMulti(p0,p1,p2:Location):Real;{tinh tich co huong}
Begin
  GetMulti:=(p1.x-p0.x)*(p2.y-p0.y)-(p2.x-p0.x)*(p1.y-p0.y);
End;
Function Intersect(Var L1,L2:Line):Boolean;{xet hai doan thang co giao nhau}
Var
  M1,M2,M3,M4:Real;
Begin
  Intersect:=False;
  If (GetMin(L1[1].x,L1[2].x)<=GetMax(L2[1].x,L2[2].x))
    And(GetMax(L1[1].x,L1[2].x)>=GetMin(L2[1].x,L2[2].x))
    And(GetMin(L1[1].y,L1[2].y)<=GetMax(L2[1].y,L2[2].y))
    And(GetMax(L1[1].y,L1[2].y)>=GetMin(L2[1].y,L2[2].y)) Then
    {phep loai tru nhanh}
  Begin

```

```

        M1:=GetMulti(L1[1],L2[1],L1[2]);M2:=GetMulti(L1[1],L1[2],L2[2]);
        M3:=GetMulti(L2[1],L1[1],L2[2]);M4:=GetMulti(L2[1],L2[2],L1[2]);
        If (Abs(M1)>Zero)And(Abs(M2)>Zero)
            And(Abs(M3)>Zero)And(Abs(M4)>Zero)
            And(M1*M2>0)And(M3*M4>0) Then Intersect:=True;
    End;
End;
Procedure GetNextPoint(Var P1,P2,P3,P4:Location);
{sap xep dinh va tinh toa do dinh thu tu}
Var
    T:Location;
Begin
    If (P2.x<P1.x)Or((P2.x=P1.x)And(P2.y<P1.y)) Then
        Begin T:=P1;P1:=P2;P2:=T;End;
    If (P3.x<P1.x)Or((P3.x=P1.x)And(P3.y<P1.y)) Then
        Begin T:=P1;P1:=P3;P3:=T;End;
    If (P3.x<P2.x)Or((P3.x=P2.x)And(P3.y<P2.y)) Then
        Begin T:=P3;P3:=P2;P2:=T;End;
    P4.x:=P1.x+P3.x-P2.x;
    P4.y:=P1.y+P3.y-P2.y;
End;
Function InBuilding(Var p0:Location):Boolean;{xet diem co nam trong hinh chu nhac}
Var
    i,j,k:Byte;
    M1,M2:Real;
    Temp:Boolean;
Begin
    k:=1;
    For i:=1 to Bld Do
        Begin
            M1:=GetMulti(p0,P[k+4],P[k+1]);
            If Abs(M1)<Zero Then Continue;
            Temp:=True;
            For j:=1 to 3 Do
                Begin
                    M2:=GetMulti(p0,P[k+j],P[k+j+1]);
                    If (Abs(M2)<Zero)Or(M1*M2<0) Then
                        Begin Temp:=False;Break;End;
                End;
            If Temp Then Begin InBuilding:=True;Exit;End;
        End;
    InBuilding:=False;
End;
Procedure Init;{doc du lieu va khoi tao}
Var
    i,j:Byte;
    Mark:Array[1..MaxPoint]Of Boolean;{danh dau cac diem can bo}
Begin
    ReadLn(Bld);
    Lne:=0;Pnt:=1;
    ReadLn(P[1].x,P[1].y,P[Bld*4+2].x,P[Bld*4+2].y);
    For i:=1 to Bld Do
        Begin
            For j:=1 to 3 Do

```



```

    Read(P[Pnt+j].x,P[Pnt+j].y);
ReadLn;
GetNextPoint(P[Pnt+1],P[Pnt+2],P[Pnt+3],P[Pnt+4]);
For j:=1 to 3 Do
    Begin
        L[Lne+j,1]:=P[Pnt+j];
        L[Lne+j,2]:=P[Pnt+j+1];
    End;
L[Lne+4,1]:=P[Pnt+4];L[Lne+4,2]:=P[Pnt+1];
L[Lne+5,1]:=P[Pnt+1];L[Lne+5,2]:=P[Pnt+3];
L[Lne+6,1]:=P[Pnt+2];L[Lne+6,2]:=P[Pnt+4];
Inc(Pnt,4);Inc(Lne,6);
End;
Inc(Pnt);
FillChar(Mark,SizeOf(Mark),False);
For i:=2 to Pnt Do
    If InBuilding(P[i]) Then Mark[i]:=True;
i:=2;
While i<Pnt Do
    Begin
        If Mark[i] Then Begin P[i]:=P[Pnt-1];P[Pnt-1]:=P[Pnt];Dec(Pnt);End;
        Inc(i);
    End;
End;
Function GetDis(a,b:Byte):Real;{tinh khoang cach giua hai diem}
Begin
    GetDis:=Sqrt((P[a].x-P[b].x)*(P[a].x-P[b].x)
        +(P[a].y-P[b].y)*(P[a].y-P[b].y));
End;
Function Visible(Var P1,P2:Location):Boolean;
{xet tu P1 co nhin thay P2 khong, tuc giua hai diem co bi toa nha chan khong}
Var
    TL:Line;
    i:Byte;
Begin
    TL[1]:=P1;TL[2]:=P2;
    For i:=1 to Lne Do
        If Intersect(L[i],TL) Then
            Begin Visible:=False;Exit;End;
    Visible:=True;
End;
Procedure Main;{chuong trinh chinh, dung gan nhan de tim duong ngan nhat}
Var
    i,j,k:Byte;
    Min:Real;
    S:Set Of Byte;
Begin
    Dis[1]:=0;
    For i:=2 to Pnt Do
        If Visible(P[1],P[i]) Then Dis[i]:=GetDis(1,i)
        Else Dis[i]:=1e10;
    S:=[1];
    For i:=2 to Pnt Do
        Begin

```

```

Min:=1e10;k:=0;
For j:=2 to Pnt Do
  If (Not(j In S))And(Dis[j]<Min) Then Begin k:=j;Min:=Dis[j];End;
If k=Pnt Then Break;
S:=S+[k];
For j:=2 to Pnt Do
  If (Not(j In S))And(Visible(P[k],P[j]))
  And(Dis[k]+GetDis(j,k)<Dis[j]) Then
    Dis[j]:=Dis[k]+GetDis(j,k);
End;
WriteLn(Dis[Pnt]:0:2);{xuat dap an}
End;
Begin
  Assign(Input,FinName);
  Reset(Input);
  Assign(Output,FoutName);
  Rewrite(Output);
  Init;
  Main;
  Close(Input);
  Close(Output);
End.

```

Tài liệu tham khảo

References

- [1] Liu Fusheng và Wang Jiande. *Hướng dẫn thi Olympic Tin học quốc tế thanh thiếu niên: tìm kiếm trí tuệ nhân tạo và thiết kế chương trình*. Nhà xuất bản Công nghiệp Điện tử, 1993.
- [2] Feng Shuchun và Xu Liutong. *Phương pháp luận thiết kế chương trình*. Nhà xuất bản Đại học Chiết Giang, 1988.
- [3] Xiang Qizhong và Wu Yaobin. *Olympic Tin học quốc tế Turbo Pascal 6.0*. Nhà xuất bản Đại học Công nghiệp Trung Nam, 1994.
- [4] Wang Jiande và Chai Xiaolu. *Phân tích đề thi lập trình đại học quốc tế*. Nhà xuất bản Đại học Phúc Đán, 1999.